

Summing an Array of Integers

We already covered summing an array of integers in Chapter 3 with the head-tail technique. In this chapter, we'll use a divide-and-conquer strategy. Since the associative property of addition means that adding $1 + 2 + 3 + 4$ is the same as adding the sums of $1 + 2$ and $3 + 4$, we can divide a large array of numbers to sum into two smaller arrays of numbers to sum.

The benefit is that for larger sets of data to process, we could farm out the subproblems to different computers and have them all work together in parallel. There's no need to wait for the first half of the array to be summed before another computer can start summing the second half. This is a large advantage of the divide-and-conquer technique, as CPUs aren't getting much faster but we can have multiple CPUs work simultaneously.

Let's ask the three questions about recursive algorithms for our summation function:

What is the base case? Either an array containing zero numbers (where we return 0) or an array containing one number (where we return the number).

What argument is passed to the recursive function call? Either the left half or the right half of the array of numbers.

How does this argument become closer to the base case? The size of the array of numbers is halved each time, eventually becoming an array containing zero or one number.

The *sumDivConq.py* Python program implements the divide-and-conquer strategy for adding numbers in the `sumDivConq()` function:

Python

```
def sumDivConq(numbers):
    if len(numbers) == 0: # BASE CASE
        ❶ return 0
    elif len(numbers) == 1: # BASE CASE
        ❷ return numbers[0]
    else: # RECURSIVE CASE
```